Assignment Cover Sheet

This cover sheet must be completed and added to the front of every assignment

Learner Name and Surname:	Imran Hossain Chowdhury
Learner Registration No.	11248
Study Centre Name	**Learn Key Institute**
Qualification Title	Undergraduate Diploma in Software Design MQF (Lv.5) — Group A
Unit Reference No.	ICT-UDSDU2-LKI/MQF (Lv.5)
Unit Title	Unit 5: Service-Oriented Architecture and Microservices Fundamentals
Assignment Title	MaltaShop Ltd: Designing and Prototyping a Microservices Platform
Submission Date	22/10/2026
Word Count	4,359 words (excluding the front page, contents, figures, captions, tables, code, references and appendices)

Declaration of authenticity:

1. **I declare that the attached submission is my own original work. No significant part of it has been submitted for any other assignment and I have acknowledged in my notes and bibliography all written and electronic sources used.**
2. **I acknowledge that my assignment will be subject to electronic scrutiny for academic honesty.**
3. **I understand that failure to meet these guidelines may instigate the centre's malpractice procedures and risk failure of the unit and / or qualification.**

Imran _____ ***Learner signature*** ***Date:***	_____ ***Tutor signature*** ***Date:***

Note:

Assignments must be submitted in typed PDF format only; handwritten assignments are not accepted.

Table of Contents

1. Architectural Evolution and Synthesis....3

1.1 Why the monolith cannot meet MaltaShop's scalability requirements....3

1.2 Monolith, ESB-based SOA or microservices....4

1.3 Distributed-systems principles and the new problems....5

2. Scalable System Design....5

2.1 Services, data and architecture....5

2.2 Design patterns and service interactions....8

2.3 Twelve-factor assessment....10

3. Lifecycle and Delivery Management....11

3.1 CI/CD pipeline for the Order service....11

3.2 Automated testing strategy for the Order service....12

3.3 Deployment strategy for the Payment service....13

4. Operational Excellence and Monitoring....17

4.1 Keeping checkout reliable at ten times normal traffic....17

4.2 Centralised logging and observability....18

4.3 Service mesh or resilience patterns first....20

5. Technical Leadership and Collaboration....21

5.1 Board briefing....21

5.2 Organising the developers and operations staff....22

5.3 Architecture Decision Records....23

6. Implementation and Prototyping....25

6.1 Keeping the place-order flow consistent....25

6.2 Dockerfile, Compose file and the path to production....29

6.3 Evidence and reflection....31

References....35

Appendix A: Git Repository....38

Appendix B: Docker Compose File....39

1. Architectural Evolution and Synthesis

1.1 Why the monolith cannot meet MaltaShop's scalability requirements

All of MaltaShop's problems come from one fact: Catalogue, Orders, Payments, Customers and Notifications are built, released, scaled and crash as a single unit. On Black Friday the only way to add checkout capacity was to run more copies of the whole application, and every copy still wrote to one MySQL database, so checkout requests queued behind browsing traffic. The March incident shows the same tight coupling at run time: a fault in a non-essential feature stopped the essential one. Table 1 maps each limitation to the case facts and to the microservices response (Lewis and Fowler, 2014; Newman, 2021).

Table 1: *Limitations of the monolith and how microservices address them*

Limitation	Evidence in the case	Microservices response
Scaling is all or nothing	8× traffic on Black Friday; checkout queued; site down for over two hours; one MySQL database on two VMs	Stateless Order, Payment and Inventory replicas scale out on their own; queues absorb bursts; catalogue reads come from a cache; each service has its own database
No fault isolation	A memory leak in e-mail notifications crashed checkout for 50 minutes	Notification runs in its own process and receives events; if it fails, orders still succeed and e-mails wait in the queue (proved in Section 6.3)
Code and data are entangled	A second payment provider took seven weeks	A Payment service with one adapter per provider behind one standardised contract, so a new provider never touches Order or Customer code
Slow, risky releases	Every three weeks, on Friday evening, by hand, 40 minutes offline	Each service is deployed separately by an automated pipeline, with no downtime
No visibility	Local log files; no dashboard or alerts; customers report incidents	Central logs, metrics, traces and alerts (Section 4.2)

The common thread is loose coupling and autonomy: each capability becomes a service that deploys, stores its data and fails on its own, so MaltaShop can scale checkout without scaling e-mails, release Payment without retesting Catalogue, and lose Notification without losing sales.

1.2 Monolith, ESB-based SOA or microservices

Table 2 compares the three options.

Table 2: *The three options compared for MaltaShop*

Criterion	Monolith (made modular)	Vendor's ESB-based SOA	Microservices
Scale checkout alone	No; the whole application scales	Partly; the ESB is a shared bottleneck	Yes
Fault isolation	Low	Medium; the ESB is a single point of failure	High, if calls are asynchronous or protected
Weekly releases	Possible with automation, but every release is the whole site	Limited; changes queue for the ESB team	Yes; each team releases its own service
Data	One shared MySQL database	Services often still share databases	Each service owns its data; others use its API or events
Where integration logic lives	In-process calls	In a central "smart" bus (Erl, 2005)	Smart endpoints, dumb pipes (Lewis and Fowler, 2014)
Governance and security	One team's rules, spread through the code	A strength: security and partner limits enforced centrally	The same policies at a thin API gateway, with no business logic in it
Cost and skills	Lowest running cost	Licence fees, proprietary skills, lock-in	Highest running cost: every service adds overhead and needs a platform team

Organisational factors decide this choice as much as technical ones. Twelve developers is a small team, which usually favours a modular monolith (Fowler, 2015), but MaltaShop's parts need very different scaling, must fail separately and have clear boundaries; twelve also make three teams of four, and Conway's law (Conway, 1968) says the system will mirror that structure. Automation could let even the monolith release weekly, but every release would still risk the whole site. Against microservices: nobody has run Kubernetes, there are only two operations staff, and GDPR and card payments demand strong governance.

An ESB is built to integrate many existing systems, but MaltaShop has one application to split, and the bus would become the new monolith: routing and transformation logic owned by one vendor and one team, with licence costs and a single point of failure. Microservices

keep SOA's standardised contracts and loose coupling (Erl, 2005) without the bus, and push autonomy further.

Recommendation: move to microservices gradually, not by a rewrite. Using the strangler fig pattern (Fowler, 2004), extract Notification first, build Recommendation and the partner API as new services, and only then move Order, Inventory and Payment; Catalogue and Customer stay in the monolith until the platform has proved itself. The target is about eight services, not hundreds, which answers the CFO's concern.

1.3 Distributed-systems principles and the new problems

Three principles shape the recommendation. **Partial failure:** some parts fail while others keep working (Rotem-Gal-Oz, 2006), so the checkout path is kept short and everything else is asynchronous; a failing Notification or Recommendation service cannot block an order. **An unreliable network:** calls time out and messages arrive twice, late or out of order, so every write carries an idempotency key and every consumer ignores events it has already processed (Section 6.1). **Eventual consistency:** without a shared database, Order, Inventory and Payment agree only after events have flowed (Vogels, 2009); MaltaShop accepts a few seconds of "order received, confirming payment".

The price is real: network latency between services, data spread over several databases with no cross-service joins, harder debugging, versioned contracts between teams, the risk of a "distributed monolith" that must still be released together, and a platform that two operations staff must learn. Sections 4 and 5 show how each is managed.

2. Scalable System Design

2.1 Services, data and architecture

MaltaShop is decomposed by business capability (Newman, 2021): each service is a bounded context that owns one area of the business and its data, so a change to that area touches one service and one team; "customer" means a profile and consent records in Customer, but only an ID in Order. Table 3 lists the eight services and Figure 1 shows how they fit together. PostgreSQL is the default store because the team already knows relational databases.

Table 3: *MaltaShop services*

Service	Responsibility	Data owned	Data store
Catalogue	Products, prices, search; publishes ProductViewed	Products, categories, prices	PostgreSQL + Redis cache
Customer	Accounts, addresses, consent, GDPR requests	Customer profiles, consent records	PostgreSQL, personal data encrypted
Order	Places orders; owns order status; starts the saga	Orders, order lines, outbox	PostgreSQL
Inventory	Stock levels and reservations	Stock, reservations	PostgreSQL
Payment	Authorises and refunds through providers, one adapter each; never stores card numbers	Payment records, provider references	PostgreSQL
Notification	E-mail, SMS and push messages triggered by events	Messages sent, templates	MongoDB (messages vary in shape)
Partner	API for the 30 retailers and the courier: stock feeds, shipment updates	Partner accounts, feed history, shipments	PostgreSQL
Recommendation	"You may also like"; runs the data team's Python model	Features, event history	Redis (serving) + object storage (history)

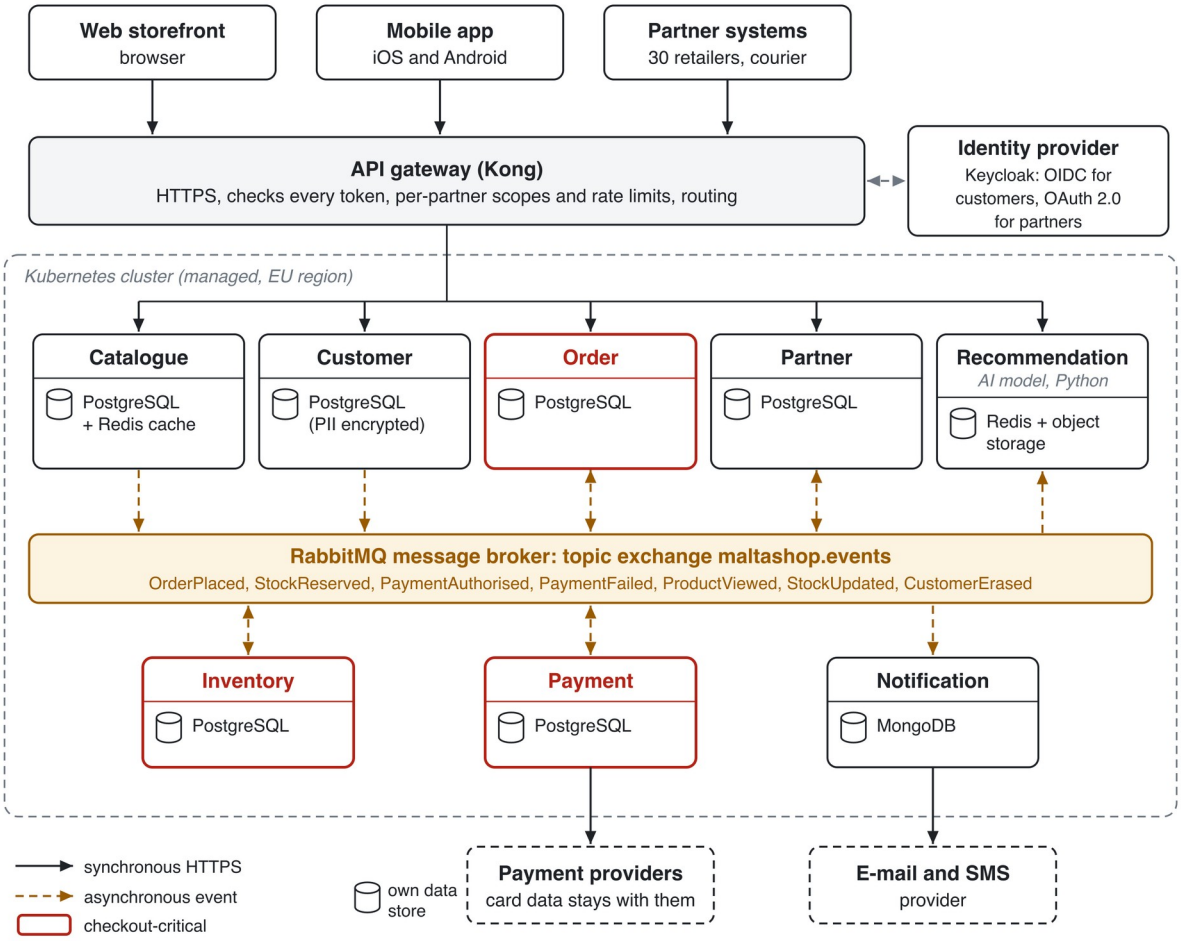


Figure 1: *Container-level architecture of the target platform. Clients reach the services only through the API gateway, which checks tokens issued by the identity provider. Checkout-critical services are outlined in red; dashed arrows are events through RabbitMQ. Each service owns its data store; to save money, low-traffic services may share one managed PostgreSQL server with a separate database and credentials each, while Order and Payment have their own.*

The AI Recommendation service sits beside checkout, not inside it. Browsing data (ProductViewed, from Catalogue) and order data (OrderPlaced) reach it as events on its own RabbitMQ queue, so it receives a copy in near real time and can fall behind or fail without anyone waiting for it. The storefront asks it for suggestions through the gateway with a 200 ms timeout and shows best-sellers if it does not answer; checkout never calls it.

Authentication happens at the gateway. Customers log in through the identity provider (OpenID Connect) and send a short-lived token with each request. Each of the 30 retailers gets its own OAuth 2.0 client with scopes such as `stock:write` or `orders:read` and its

own rate limit. Services check the scopes again, so one gateway mistake does not expose data.

2.2 Design patterns and service interactions

Table 4: *Design patterns and why they were chosen*

Pattern	Where	How it keeps services decoupled and robust
API gateway	In front of all services	One entry point for web, app and partners; authentication, rate limits and routing in one place; kept thin (no business rules) and replicated, so it cannot become a new ESB
Database per service	Every service	No shared schema, so teams change their own tables freely; removes the shared-MySQL bottleneck; enforced by credentials and network rules
Publish–subscribe	OrderPlaced, StockReserved, ProductViewed and others	A topic exchange gives each subscriber its own durable queue; producers do not know their consumers, so Recommendation is added without changing Order
Saga with transactional outbox	Place-order flow	Keeps Order, Inventory and Payment consistent without a distributed transaction (Section 6.1)
Timeout, retry with backoff, circuit breaker, fallback	Gateway to Recommendation; Payment to providers	Stops waiting on a failing dependency, retries only idempotent calls, and returns a fallback such as best-sellers (Nygard, 2018)
Strangler fig	Migration	The gateway moves one route at a time from the monolith to a new service; rolling back means routing back

The patterns in Table 4 come from Richardson (2018), but the case decides which ones matter: publish–subscribe answers the March incident and the AI requirement, database per service removes the MySQL bottleneck, and the strangler fig limits the risk of migration for a team new to microservices. Table 5 states whether each interaction is synchronous or

asynchronous. The rule is simple: a call is synchronous only when the caller cannot answer its own user without the reply, because five chained calls at 99% each would leave checkout at about 95%.

Table 5: *Interaction matrix. REST serves the clients; no internal call waits on another, so gRPC is not needed*

From → to	Interaction	Style	Why
Storefront → Catalogue	Browse products	Sync (REST)	The customer is waiting for the page; answers are cached
Storefront → Order	Place order	Sync (REST); replies PENDING	The customer needs an order number at once
Order → Inventory	OrderPlaced	Async (pub/sub event)	Reserving stock must not hold the customer's request open
Inventory → Payment, Order	StockReserved / StockReservationFailed	Async (event)	Payment is taken only once stock is held; otherwise the order is cancelled
Payment → Order, Inventory	PaymentAuthorised / PaymentFailed	Async	The saga's outcome: confirmation or compensation
Payment → payment provider	Authorise, refund	Sync (HTTPS), timeout and circuit breaker	External API; the idempotency key is the order ID
Order, Payment → Notification	Order events	Async (replicas share one queue)	An e-mail must never slow or break checkout
Catalogue, Order → Recommendation	ProductViewed, OrderPlaced	Async (own queue)	Near-real-time data without coupling to checkout
Storefront → Recommendation	Get suggestions	Sync, 200 ms timeout, fallback	Best-sellers are shown if it is slow
Partner → Inventory	StockUpdated	Async (work queue)	Retailer feeds arrive in bursts; the queue smooths them
Customer → all	CustomerErased	Async	A GDPR erasure reaches every service holding personal data

2.3 Twelve-factor assessment

Table 6 assesses the design against eight of the twelve factors (Wiggins, 2017), with the evidence in the prototype.

Table 6: *Twelve-factor assessment*

Factor	How the design satisfies it	Evidence in the prototype
III Config	All settings come from environment variables; secrets from a secrets manager in production	`config.py` reads only environment variables and holds no passwords
IV Backing services	Databases and the broker are attached resources addressed by URL, so a local container can become a managed service without code changes	`DATABASE_URL`, `MONGO_URL`, `RABBITMQ_URL`
V Build, release, run	One image per commit; a release is that image plus environment configuration	CI builds one image per commit and tests that same image
VI Processes	Services keep no state in memory; any replica can serve any request	The outbox is in PostgreSQL, not in memory
VIII Concurrency	Scale out, not up: checkout adds identical replicas for a sale (Section 4.1)	`FOR UPDATE SKIP LOCKED` lets several outbox relays run side by side (Listing 5)
IX Disposability	Fast start, safe stop: unacknowledged messages are redelivered and unsent outbox rows are retried	Manual acknowledgements; the resilience test in Section 6.3
X Dev/prod parity	The same images and the same database and broker engines everywhere	Compose runs PostgreSQL, MongoDB and RabbitMQ, as production will
XI Logs	Logs are an event stream to standard output, collected by the platform	JSON logs to stdout, read with `docker compose logs`

Twelve-factor services are what cloud-native infrastructure expects. Kubernetes can start, stop and replace any replica because none holds state (VI, IX); the autoscaler can add replicas because services scale out, not up (VIII); and the platform, not the code, decides where logs go (XI). The main gap is admin processes (XII): the prototype creates its tables at start-up, and this must become a versioned migration job before production.

3. Lifecycle and Delivery Management

3.1 CI/CD pipeline for the Order service

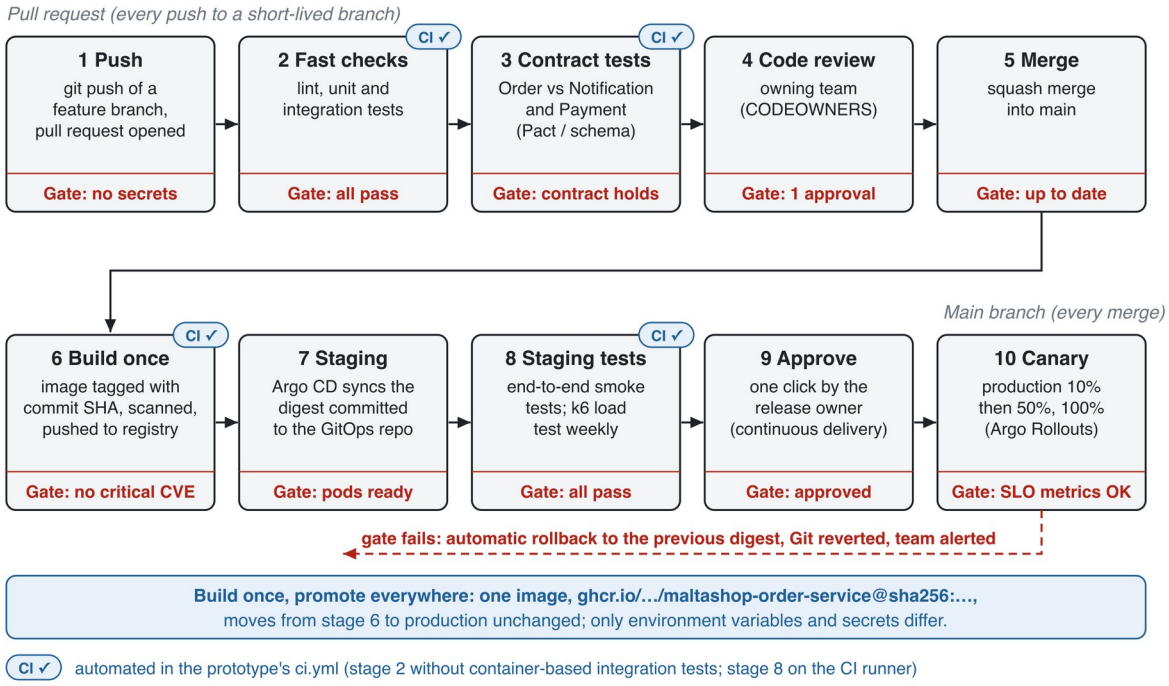


Figure 2: *The CI/CD pipeline for the Order service, from a developer's git push to production. Each stage has a gate (red) that must pass before the change moves on. Blue tags mark the stages already automated in the prototype's GitHub Actions workflow.*

Figure 2 has two halves. The pull-request half gives the developer feedback within minutes: lint, unit and integration tests (stage 2), then contract tests (stage 3), which fail the build if Order would break a consumer such as Notification. A reviewer from the owning team must approve (stage 4) before the branch is merged.

The main-branch half produces and promotes the release. Stage 6 builds the container image once, tags it with the commit SHA, scans it and pushes it to the registry. From then on the image is never rebuilt: Argo CD deploys the same digest to staging, the end-to-end smoke tests (and a weekly load test) run against it, and after approval the same digest goes to production as a canary. This is "build once, promote everywhere" (Humble and Farley, 2010): what was tested is exactly what runs, environments differ only in configuration, and rolling back means redeploying the previous digest. The prototype already works this way: its workflow tests the images it has just built, without rebuilding them (Figure 3).

Branching follows trunk-based development (Hammant, 2017): short-lived feature branches merged at least daily, with unfinished work hidden behind feature flags. Long-lived branches would bring back the large, risky merges of the three-week release train. The pipeline is itself an attack surface, so third-party actions are pinned to commit SHAs, each job gets only the permissions it needs, push protection blocks committed secrets, and only the pipeline can push images.

MaltaShop should choose **continuous delivery**, not continuous deployment, for now. Every merge produces a release candidate that could go live, but a person approves the production step (stage 9) for Order and Payment, because they handle money and the team is new to the platform. Once three months of DORA metrics show few failed changes, low-risk services such as Notification and Catalogue can move to continuous deployment.

3.2 Automated testing strategy for the Order service

Testing follows the test pyramid (Cohn, 2009; Vocke, 2018): many fast unit tests at the base and fewer, slower tests towards the top. Table 7 sets out what each level checks in the Order service and where it runs in Figure 2.

Table 7: *Test pyramid for the Order service*

Level	What is tested	Tools	Pipeline stage
Unit (many, milliseconds)	Pricing and totals; validation (unknown SKU, quantity 1–20); status changes PENDING → CONFIRMED or CANCELLED; events carry no personal data	pytest	2, every push
Integration (some, seconds)	The API with a real PostgreSQL and RabbitMQ in containers: idempotency key, outbox written in the same transaction, relay publishes	pytest, Testcontainers (planned)	2, every push
Contract (one per pair)	OrderPlaced still meets Notification's needs; Order's calls still match Payment's API	JSON Schema now, Pact later	3, before merge
End-to-end (few)	The whole place-order flow across services	`smoke_test.py`	8, staging (prototype: CI runner)
Load (weekly, before sales)	Ten times normal checkout traffic; p95 latency and error rate	k6	8, weekly

The contract test in Listing 1 is between two named services: **Order** (provider) and **Notification** (consumer). The fields that Notification relies on are written down in `contracts/order-placed.v1.schema.json`. Order's build fails if its event no longer satisfies them, and Notification's build checks it can handle the contract's example.

Listing 1: *Provider side of the OrderPlaced contract test (order-service/tests)*

```
def test_order_placed_event_satisfies_the_consumer_contract():
    lines = [OrderLine(sku="TV-55-4K", quantity=1),
             OrderLine(sku="KETTLE-1.7L", quantity=2)]
    order = {"order_id": uuid.uuid4(), "customer_id": "C-1001",
             "status": "PENDING", "total_amount": order_total(lines),
             "currency": "EUR", "items": price_lines(lines)}

    event = json.loads(json.dumps(
        order_placed_event(order, correlation_id=str(uuid.uuid4()))))

    # raises ValidationError if the event breaks the consumer's contract
    Draft202012Validator(SCHEMA, format_checker=FormatChecker()).validate(event)
```

Renaming `customer_id` made this test fail ("'customer_id' is a required property"), which is exactly the breaking change it exists to stop. Production would use Pact (Pact Foundation, n.d.) and its can-i-deploy check.

3.3 Deployment strategy for the Payment service

Table 8: *Rolling, blue-green and canary deployment for Payment*

Strategy	How it works	Strengths for Payment	Weaknesses for Payment
Rolling	Replaces pods a few at a time (the Kubernetes default)	No extra capacity; simple	No control over who gets the new version; a bad release reaches everyone gradually; rollback is another slow rollout
Blue-green	A full new copy runs beside the old one; all traffic switches at once (Fowler, 2010)	Instant switch and instant rollback	Double capacity; both versions share one database; every customer meets the new version at once
Canary	A small share of traffic goes to the new version and grows while metrics stay healthy (Sato, 2014)	Limits damage to a few payments; decisions based on real data	Needs good metrics and automation (Argo Rollouts); slower

Of the options in Table 8, **canary** is the right choice for Payment. A faulty payment release costs money directly and damages trust, so exposure should be limited and judged on real data. The canary sends 10% of payment traffic to the new version for 15 minutes, then 50%, then 100%. At each step Argo Rollouts compares the canary with the stable version on payment error rate, provider decline rate and p95 latency. The rollback procedure is:

1. If any metric breaches its threshold, Argo Rollouts aborts automatically, sends all traffic back to the stable version within seconds and pages the on-call engineer.
2. A payment retried during the switch cannot be charged twice, because the provider receives the order ID as its idempotency key.
3. Database changes follow expand-and-contract (new columns first, old ones removed in a later release), so the previous version still works with the current schema.
4. The engineer reverts the digest commit in the GitOps repository so that Git matches production, confirms recovery on the dashboard, and the fix goes through the normal pipeline; rollback is rehearsed monthly.

Two DORA metrics (Forsgren, Humble and Kim, 2018) will show whether delivery improves (Table 9). Change failure rate has no baseline yet, because failures are not recorded; tracking starts with the first pipeline release.

Table 9: *DORA metrics: baseline and target*

Metric	MaltaShop baseline (case facts)	Target within 12 months
Deployment frequency	Once every three weeks, whole site, 40 minutes offline	At least weekly per service with no downtime; daily for low-risk services
Time to restore service	50 minutes (March) to over two hours (Black Friday); customers notice first	Under 30 minutes for checkout, with an alert before customers notice

Evidence: pipeline workflow file and a passing run

Listing 2 is the prototype's `.github/workflows/ci.yml`. Pull requests run the tests; every push to `main` also builds each image once, scans it with Trivy, pushes it and runs an end-to-end test of those exact images.

Listing 2: *The prototype's CI workflow (.github/workflows/ci.yml)*

```
name: CI

# Pull requests run stage 1. Every push to main also builds each image ONCE, tagged with the
# commit SHA, scans it, pushes it, and the end-to-end stage tests those exact images
```

```
# ("build once, promote everywhere"). Actions are pinned to a full commit SHA (the comment
# gives the version) so a moved or hijacked tag cannot change what runs in the pipeline.
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

permissions:
  contents: read

env:
  IMAGE_REGISTRY: ghcr.io/${{ github.repository_owner }}
  IMAGE_TAG: ${{ github.sha }}

jobs:
  test:
    name: "Stage 1: lint, unit and contract tests (${{ matrix.service }})"
    runs-on: ubuntu-24.04
    strategy:
      matrix:
        service: [order-service, notification-service]
    defaults:
      run:
        working-directory: ${{ matrix.service }}
    steps:
      - name: Check out repository
        uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1  # v7.0.1

      - name: Set up Python
        uses: actions/setup-python@5fda3b95a4ea91299a34e894583c3862153e4b97  # v7.0.0
        with:
          python-version: "3.12"
          cache: pip
          cache-dependency-path: ${{ matrix.service }}/requirements*.txt

      - name: Install dependencies
        run: pip install -r requirements.txt -r requirements-dev.txt

      - name: Lint (gate)
        run: ruff check app tests

      - name: Unit and contract tests (gate)
        run: pytest -v

  build:
    name: "Stage 2: build, scan and push image (${{ matrix.service }})"
    needs: test
    if: github.event_name == 'push'
    runs-on: ubuntu-24.04
    permissions:
      contents: read
      packages: write
    strategy:
      matrix:
        service: [order-service, notification-service]
    env:
      IMAGE: ghcr.io/${{ github.repository_owner }}/maltashop-${{ matrix.service }}:$
  {{ github.sha }}
    steps:
      - name: Check out repository
        uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1  # v7.0.1

      - name: Log in to GitHub Container Registry
        uses: docker/login-action@dbcb813823bdd20940b903addbd779551569679f  # v4.6.0
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - name: Build once, tagged with the commit SHA
        uses: docker/build-push-action@c3c9e263c25d99ce0380d002d59b67737d91b0dc  # v7.4.0
        with:
          context: ${{ matrix.service }}
          load: true
          tags: ${{ env.IMAGE }}

      - name: Scan the image; stop on any critical vulnerability that has a fix (gate)
        run: >
          docker run --rm -v /var/run/docker.sock:/var/run/docker.sock
          aquasec/trivy:0.76.0@sha256:2416d1f74045b57274df1a340eee6b510cacd73c66a6ccb8da092b117eb7d
  222
```

```
        image --severity CRITICAL --ignore-unfixed --exit-code 1 --no-progress "$IMAGE"

    - name: Push the scanned image
      run: docker push "$IMAGE"

end-to-end:
  name: "Stage 3: end-to-end test of the built images"
  needs: build
  runs-on: ubuntu-24.04
  permissions:
    contents: read
    packages: read
  steps:
    - name: Check out repository
      uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1  # v7.0.1

    - name: Log in to GitHub Container Registry
      uses: docker/login-action@dbcb813823bdd20940b903addbd779551569679f  # v4.6.0
      with:
        registry: ghcr.io
        username: ${{ github.actor }}
        password: ${{ secrets.GITHUB_TOKEN }}

    - name: Pull the images built in stage 2 (no rebuild)
      run: docker compose pull order-service notification-service

    - name: Start the whole system and wait until every container is healthy
      run: docker compose up -d --no-build --wait

    - name: Smoke test the place-order flow (gate)
      run: python3 scripts/smoke_test.py

    - name: Show service logs
      if: always()
      run: docker compose logs order-service notification-service

    - name: Stop the system
      if: always()
      run: docker compose down -v
```

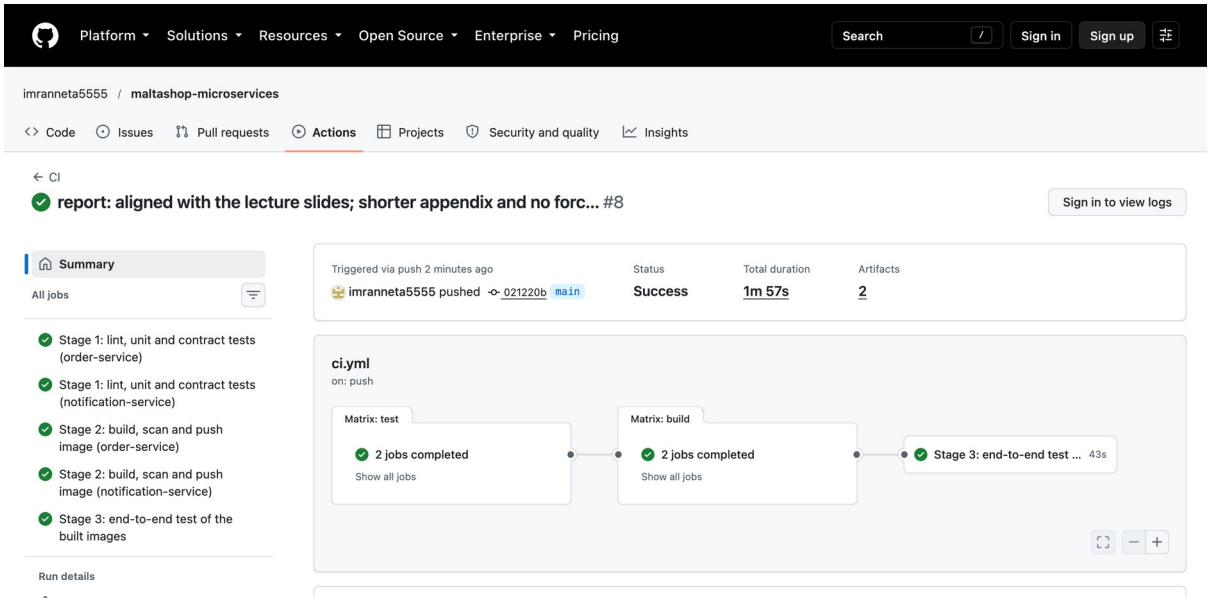


Figure 3: *A passing run of the prototype's CI workflow on GitHub Actions, in 1 min 57 s: well inside a ten-minute target, because both services are tested in parallel and pip downloads are cached. Stage 1 lints and tests both services; stage 2 builds each image once, scans it with Trivy and pushes it to GitHub Container Registry; stage 3 starts the whole system from those images and runs the end-to-end smoke test.*

4. Operational Excellence and Monitoring

4.1 Keeping checkout reliable at ten times normal traffic

Kubernetes keeps checkout reliable by constantly making the cluster match a declared state (Burns *et al.*, 2022). Order, Payment and Inventory each run as a **Deployment**, which manages a **ReplicaSet**: if a pod crashes or its node fails, the ReplicaSet starts a replacement. Pods are spread across three availability zones, so losing a zone leaves two-thirds of the capacity.

Probes decide which pods receive traffic (Kubernetes, n.d.a). The readiness probe calls `/health/ready`, which checks the database; a pod that is starting, overloaded or cut off is removed from the Service instead of returning errors. The liveness probe calls `/health/live`, which checks nothing else, and restarts only a pod that hangs. **Resource requests** reserve CPU and memory so the scheduler only places pods where they fit, and **limits** cap each pod: the March memory leak would now restart one Notification pod, not take down checkout.

For the peak, the **Horizontal Pod Autoscaler** adds Order replicas when average CPU use passes 60%, from 3 up to 30 (Kubernetes, n.d.b), and the cluster autoscaler adds nodes when pods do not fit. Because autoscaling takes minutes, the minimum is raised before a planned sale, after a k6 test at ten times normal traffic on staging. Listing 3 shows these settings for the Order service.

Listing 3: *Order service Deployment and autoscaler (excerpt)*

```
apiVersion: apps/v1
kind: Deployment
metadata: {name: order-service}
spec:
  replicas: 3
  selector: {matchLabels: {app: order-service}}
  template:
    metadata: {labels: {app: order-service}}
    spec:
      topologySpreadConstraints:          # spread replicas over three zones
        - maxSkew: 1
          topologyKey: topology.kubernetes.io/zone
          whenUnsatisfiable: ScheduleAnyway
          labelSelector: {matchLabels: {app: order-service}}
      containers:
        - name: order-service
          # the exact image digest that passed the pipeline
          image: ghcr.io/imranneta5555/maltashop-order-service@sha256:...
          envFrom:                        # settings: ConfigMap; passwords: Secret
            - configMapRef: {name: order-service-config}
            - secretRef: {name: order-service-secrets}
          securityContext: {runAsNonRoot: true, readOnlyRootFilesystem: true}
```

```
          ports: [{containerPort: 8000}]
          resources:
            requests: {cpu: 250m, memory: 256Mi}
            limits: {memory: 512Mi}       # a leak ends in a restart
          readinessProbe:
            httpGet: {path: /health/ready, port: 8000}
            periodSeconds: 5
            failureThreshold: 2
          livenessProbe:
            httpGet: {path: /health/live, port: 8000}
            periodSeconds: 10
            failureThreshold: 3
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: {name: order-service}
spec:
  scaleTargetRef: {apiVersion: apps/v1, kind: Deployment, name: order-service}
  minReplicas: 3
  maxReplicas: 30
  metrics:
    - type: Resource
      resource: {name: cpu, target: {type: Utilization, averageUtilization: 60}}
```

All eight services and the gateway are stateless and run as Deployments. The stateful parts (PostgreSQL, MongoDB, Redis and RabbitMQ) stay outside the cluster as managed services, such as Amazon RDS with a standby in a second zone, so the team runs no database StatefulSets in its first year.

Because nobody has run Kubernetes in production, the risk is reduced in steps: a managed control plane (Amazon EKS) in an EU region; all cluster configuration in Git, applied by Argo CD; training for the operations staff and one developer per team; an external specialist for set-up and the first three months of on-call; and Notification as the first service in production, because its failure does the least harm.

4.2 Centralised logging and observability

Today each server writes its own log file and nobody is alerted. Now every service must write one JSON object per line to standard output with the fields in Table 10, as the prototype already does (Figure 10).

Table 10: *Mandatory log schema*

Field	Example	Rule
`timestamp`	2026-10-05T04:28:58.666Z	UTC, ISO 8601, milliseconds
`level`	INFO	DEBUG, INFO, WARNING or ERROR
`service`, `version`, `environment`	order-service, 1f3c9a2, production	Identify what logged it and which release

Field	Example	Rule
`correlation_id`	8f2bdf1c-d1c3-…	From the X-Correlation-ID header, or created at the gateway; copied into every event and outgoing call
`trace_id`, `span_id`	4bf92f3577b34da6…	From OpenTelemetry; links the log line to its trace
`event`	order.placed	Stable, searchable name
`message`	Order saved as PENDING	For people
Context	`order_id`, `event_id`, `duration_ms`	IDs only: no names, e-mail addresses or card data (European Parliament and Council of the European Union, 2016; PCI Security Standards Council, 2022)

Every service is instrumented with OpenTelemetry (OpenTelemetry, n.d.), and a Collector on each node removes personal-data fields and samples traces. Logs go to Loki, metrics to Prometheus and traces to Tempo, all viewed in Grafana and hosted by Grafana Cloud in an EU region (Figure 4; ADR-002 in Section 5.3).

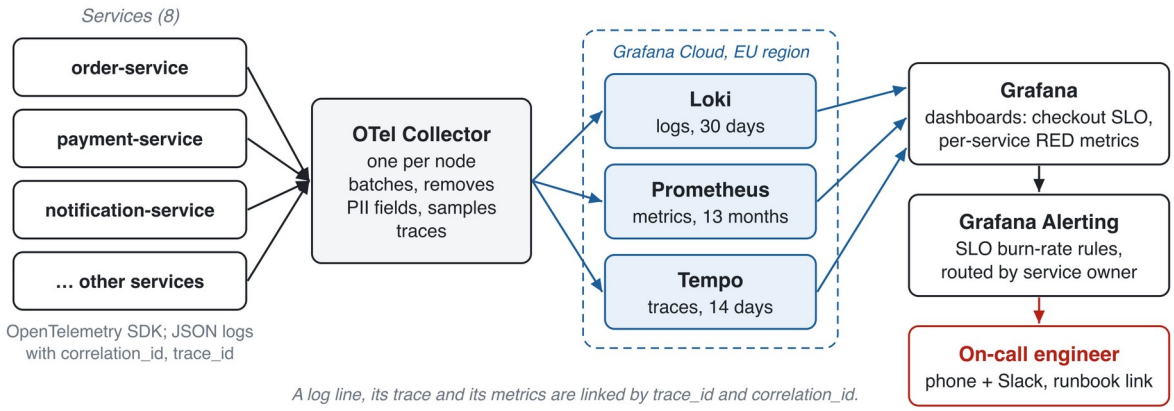


Figure 4: *The observability pipeline. Services send JSON logs, metrics and traces through an OpenTelemetry Collector to Loki, Prometheus and Tempo; Grafana shows them together and its alert rules page the on-call engineer of the owning team, with a link to the runbook.*

Checkout SLO: 99.9% of checkout requests (placing an order and confirming payment) succeed, without a server error and within 2 seconds, measured at the gateway over a rolling 30 days. This leaves an error budget of 0.1%, about 43 minutes of failure a month. The two alerts in Table 11 use the burn-rate method (Beyer *et al.*, 2018).

Table 11: *Checkout alerts*

Alert	Metric	Threshold and duration	Severity	First runbook step
Checkout fast burn	Share of failed or slow checkout requests	Over 1.44% (budget burning 14.4× too fast) for both the last hour and the last 5 minutes	Critical: page on-call	Open the checkout dashboard; if anything was deployed in the last hour, roll it back first
Checkout slow burn	Same	Over 0.6% (6× too fast) for both the last 6 hours and the last 30 minutes	Warning: ticket for the owning team	Find which service's error rate rose on the RED dashboard, then search its logs by correlation ID

Replaying the **March incident** with this design shows the difference:

1. Notification's memory grows after a release. Kubernetes restarts each pod at its 512 Mi limit, and an alert fires on more than three restarts in 15 minutes. Checkout is unaffected: orders are accepted and their events wait in RabbitMQ (Section 6.3), so the checkout alerts stay silent.
2. The Notification dashboard shows memory climbing from the moment version 1.8 was deployed (deployments are marked on the graphs).
3. In Loki the engineer filters Notification's ERROR lines and finds out-of-memory errors in the template renderer; following one correlation ID to its trace shows rendering time growing with every message.
4. They roll Notification back to the previous image digest. The waiting events are then processed, and because the consumer ignores repeated event IDs, no customer receives a duplicate e-mail.

Detection takes minutes instead of waiting for complaints, and the failure costs e-mails, not sales.

4.3 Service mesh or resilience patterns first

A service mesh places a sidecar proxy (Envoy, in Istio) next to every pod, so all traffic between services passes through proxies that are configured centrally (Istio, n.d.). It adds fault tolerance without code changes: **mutual TLS** encrypts and authenticates every call; **retries** (with a budget) and **timeouts** are set per route; **circuit breaking** ejects failing

instances; and **traffic shifting** sends, for example, 10% of calls to a new version, which is how canary releases are routed.

MaltaShop should **not adopt a mesh now**, but build resilience into the services and the gateway first. Most traffic between services goes asynchronously through RabbitMQ, which a mesh does not manage. The few synchronous calls (gateway to services, Payment to its providers) are protected by timeouts, retries with idempotency keys and circuit breakers (Nygard, 2018), and Argo Rollouts already splits canary traffic at the ingress controller. A mesh would add a proxy to every pod, more memory and a complex control plane for two operations staff who are still learning Kubernetes. Revisit this after twelve months, or sooner if an auditor requires encryption inside the cluster; Linkerd would then be the lighter option.

5. Technical Leadership and Collaboration

5.1 Board briefing

To: the CEO and CFO. **From:** the Lead Solutions Architect. **Subject:** replacing the MaltaShop platform.

What I recommend. Today the shop is one large program: when one part fails or gets busy, everything fails: two hours offline last Black Friday, 50 minutes in March because of the e-mail feature, and 40 minutes offline for every update. I recommend splitting it, step by step, into about eight smaller parts, such as checkout, payments and e-mails, that run and fail separately.

What we gain.

- Checkout keeps working at ten times normal traffic, because we can strengthen checkout alone.
- A fault in e-mails or recommendations no longer stops sales.
- Weekly updates with no downtime (updates alone now cost about 11 hours offline a year).
- AI recommendations, 30 partner connections and new payment providers take weeks, not months.

What it costs (estimates). About €2,500 a month more for rented cloud services and monitoring, and about €30,000 in the first year for training and an outside expert. We rent ready-made services and avoid the integration vendor's licence fees.

The CFO's concern. It is right: this approach is overkill for small companies that rewrite everything into hundreds of parts. We will not do that. We split off only the parts causing our problems, one at a time, while the current shop keeps running. After the first part (e-mails, about three months) the board reviews the results and can stop with little lost.

Risks. The team must learn new tools, so we pay for training and help. For a while we pay for old and new systems together, and other new features come more slowly. More parts mean more to watch, so monitoring comes first.

Keeping the current system costs less this year, but cannot meet next year's goals.

5.2 Organising the developers and operations staff

Before the first service is extracted, the twelve developers become three stream-aligned teams of four, each owning whole business capabilities, and the two operations staff become a small platform team (Skelton and Pais, 2019):

- **Checkout team:** Order, Payment and Inventory.
- **Storefront team:** Catalogue, Customer, and Recommendation, built with the data team.
- **Integrations team:** Partner API, Notification, gateway routes, and the remaining monolith until it is retired.
- **Platform team (two operations staff):** the cluster, pipeline templates, observability, and second-line on-call support.

Service ownership. Every service has exactly one owning team, recorded in a service catalogue and a CODEOWNERS file. That team builds it, runs it and is paged for it.

Git workflow and code review. Trunk-based development with pull requests; one approval from the owning team and a green pipeline before merging; a change to another team's service is reviewed by that team.

API contracts. REST APIs are described in OpenAPI and events in JSON Schema, stored with the code. A breaking change needs a new version and the old one is kept for at least six months; contract tests enforce this (Section 3.2).

Shared documentation. Each repository has a README, runbooks and Architecture Decision Records, linked from the service catalogue.

On-call and post-incident review. Each team has a weekly business-hours rota for its services, with the platform team as second line, and one shared rota out of hours; alerts are

routed by owner label and link to a runbook. Reviews are blameless, held within five working days, built on a timeline from logs and traces, and their actions are tracked to completion. Table 12 lists three operational complexities this way of working creates, and how each is managed.

Table 12: *Operational complexities and how each is managed*

Complexity	Strategy
Debugging across team boundaries: one order passes through services owned by two or three teams	Mandatory correlation IDs and tracing (Section 4.2); one shared checkout dashboard; the team that owns the failing SLO leads the incident
Contract drift: one team changes an event or API and breaks another team's service	Consumer-driven contract tests in CI, a versioning policy, and contract changes reviewed by both teams
On-call load and knowledge silos: fourteen people now run eight services, more than two operations staff can carry	"You build it, you run it" rotas, a runbook for every alert, shared pipeline and deployment templates, and new on-call engineers paired with experienced ones

5.3 Architecture Decision Records

The two ADRs follow Nygard's (2011) format.

ADR-001: Message broker: RabbitMQ (managed). Status: accepted, October 2026.

Context. Order, Inventory, Payment, Notification and Recommendation must exchange events reliably. Even at ten times normal traffic the volume is modest (assumption: under 200 events per second), the operations staff have no event-streaming experience, and the broker must run on a laptop as well as in production.

Decision. RabbitMQ with topic exchanges, durable queues, publisher confirms and dead-letter queues; in production, managed RabbitMQ (Amazon MQ) in an EU region. Criteria: operational effort, team skills, cost, delivery guarantees, replay and lock-in; Table 13 shows the alternatives rejected.

Table 13: *ADR-001: alternatives rejected*

Alternative	Why rejected
Apache Kafka (managed)	Better replay and throughput, but more concepts to learn (partitions, offsets) and a higher cost than MaltaShop's volume justifies

Alternative	Why rejected
The vendor's ESB	Licence cost, a central team and business logic in the bus (Section 1.2)
Amazon SNS and SQS	Cheap and simple, but ties every service to one cloud provider and cannot run in Docker Compose for development

Consequences. RabbitMQ is well known, runs identically in the prototype and in production, and its per-message acknowledgements fit the saga. Queues delete messages once consumed, so Recommendation keeps its own event history. *Technical debt accepted:* high-volume streaming or replay for many consumers would need RabbitMQ Streams or Kafka; publishing sits in one small module per service to contain that change.

ADR-002: Observability stack: OpenTelemetry with Grafana Cloud. Status: accepted, October 2026.

Context. Logs are local files and customers report incidents. The team needs logs, metrics, traces and alerts in one place, on a limited budget, run by two operations staff, with personal data kept in the EU.

Decision. Instrument every service with OpenTelemetry and use Grafana Cloud (Loki, Prometheus, Tempo, Grafana Alerting) in an EU region. Criteria: cost at MaltaShop's volume, operational effort, vendor neutrality, all three signals in one tool, and EU data residency; Table 14 shows the alternatives rejected.

Table 14: *ADR-002: alternatives rejected*

Alternative	Why rejected
Self-hosted Elastic (ELK) stack	Powerful search, but running Elasticsearch clusters needs time and skills the operations staff do not have
Datadog	Excellent, but per-host and per-gigabyte pricing grows fast and its agents increase lock-in
Cloud provider tools only (CloudWatch)	Cheaper, but weaker tracing and dashboards, and tied to one provider

Consequences. There is nothing to operate, and OpenTelemetry lets the backend change without touching service code. The bill grows with log volume, so production logs exclude DEBUG, are kept 30 days, and only 10% of normal traces are stored (all error traces are kept). *Technical debt accepted:* dashboards and alerts are first built by hand and must later move into code (Terraform) to be reviewed and versioned like the services.

Effect on the project. Both decisions favour tools the team can run now over the most powerful option, which raises the chance of meeting the 12-month goals: MaltaShop's scarcest resource is operations time, not technology. The debt is written down, so future teams know what was traded and when to revisit it.

6. Implementation and Prototyping

The prototype implements the place-order flow: the **Order service** (FastAPI, PostgreSQL) exposes `POST /orders`, and the **Notification service** (FastAPI, MongoDB) consumes the `OrderPlaced` event through **RabbitMQ**. Each has a Dockerfile, one `docker compose up` starts everything, and the README explains how to run it and its environment variables (Appendix A).

6.1 Keeping the place-order flow consistent

An order touches three databases (Order, Inventory and Payment), so one ACID transaction is impossible, and a two-phase commit would lock all three and fail whenever one is down. The flow is therefore a **saga** (Garcia-Molina and Salem, 1987; Richardson, 2018): a sequence of local transactions, each publishing an event that triggers the next, with a compensating action that undoes a step if a later one fails (Figure 5).

I chose **choreography** rather than orchestration. With three participants and one main path, the events are easy to follow and no orchestrator service has to be built and run, while Order still records the state of every order. If the flow grows to include returns or split deliveries, an orchestrator would become easier to manage.

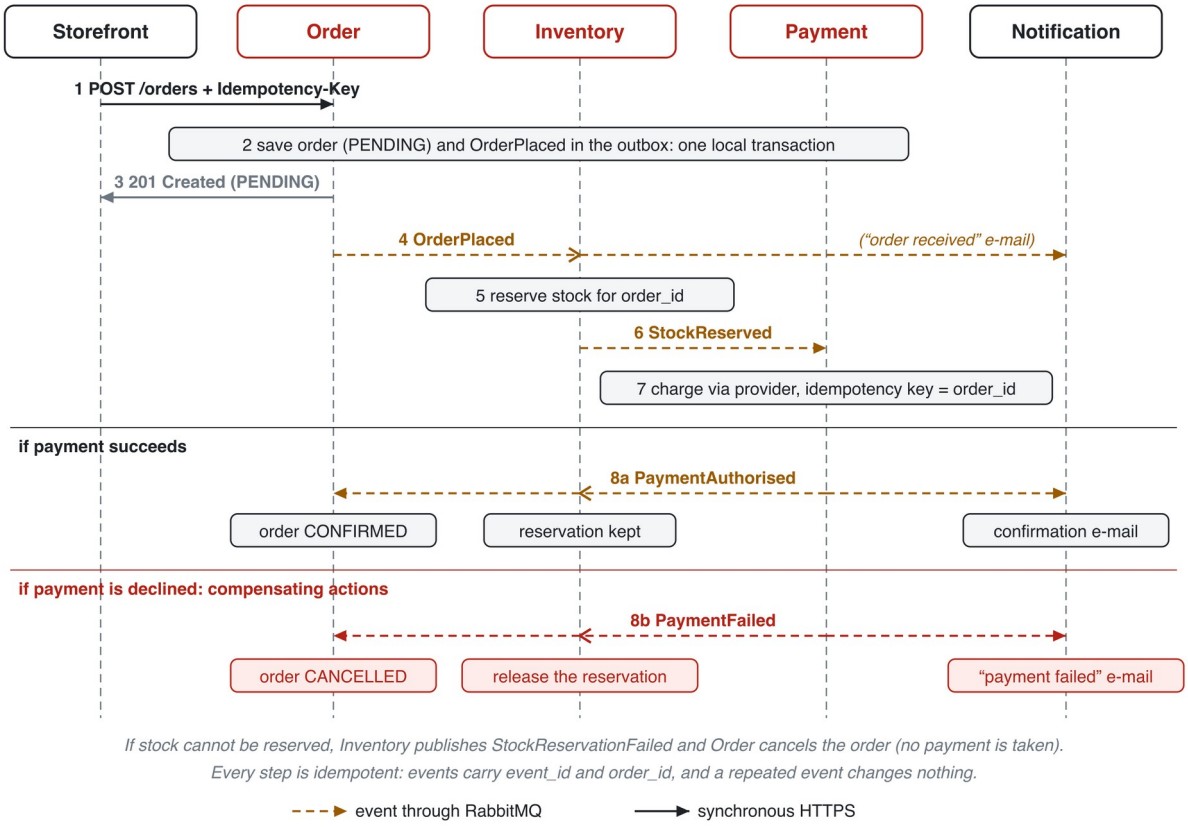


Figure 5: *The place-order saga (choreography). Each service commits a local transaction and publishes an event; the red path shows the compensating actions taken when payment is declined. Only Order (steps 1–4) and Notification are built in the prototype.*

Three techniques make every step safe to repeat:

1. **Idempotent API.** `POST /orders` takes an `Idempotency-Key` header. The key is a unique column, so a retried request returns the original order and writes nothing (Listing 4; Figure 11).
2. **Transactional outbox.** The order and its event are saved in the same local transaction, so there is never an order without its event or an event without its order (Richardson, n.d.). A relay publishes the event and marks it sent only after RabbitMQ confirms it (Listing 5); a crash in between causes a duplicate, never a loss.
3. **Idempotent consumer.** The consumer acknowledges a message only after its work is saved, and a unique index on `event_id` makes a repeated event harmless (Listing 6).

The framework and broker do much of this work. PostgreSQL provides the transaction, the unique constraint and `ON CONFLICT DO NOTHING`; RabbitMQ provides durable queues, persistent messages, publisher confirms, manual acknowledgements and dead-letter queues

(RabbitMQ, n.d.); and FastAPI's pydantic models reject an invalid order before anything is written.

Listing 4: *Idempotent order creation with the transactional outbox (order-service/app/db.py, annotated excerpt)*

```
# (1) one local PostgreSQL transaction for the order AND its event
with connect() as conn, conn.transaction():
    order = conn.execute(
        """INSERT INTO orders (order_id, idempotency_key, customer_id, status,
                               total_amount, currency, items, correlation_id)
           VALUES (gen_random_uuid(), %s, %s, 'PENDING', %s, %s, %s, %s)
           ON CONFLICT (idempotency_key) DO NOTHING
           RETURNING *""",
        (idempotency_key, request.customer_id, order_total(request.items),
         CURRENCY, Jsonb(price_lines(request.items)), correlation_id),
    ).fetchone()
    # (2) the key was used before: nothing was written, return the original
    if order is None:
        existing = conn.execute("SELECT * FROM orders WHERE idempotency_key = %s",
                                (idempotency_key,)).fetchone()
        return existing, False

    # (3) the OrderPlaced event goes into the outbox in the same transaction
    event = order_placed_event(order, correlation_id)
    conn.execute(
        "INSERT INTO outbox (event_id, event_type, routing_key, payload) "
        "VALUES (%s, %s, %s, %s)",
        (event["event_id"], event["event_type"], "order.placed", Jsonb(event)),
    )
    # (4) commit: both rows are saved, or neither is
    return order, True
```

Listing 5: *The outbox relay publishes with confirms (order-service/app/outbox.py, annotated excerpt; logging omitted)*

```
# (1) take unsent events and lock them, so two relays never send the same row
rows = conn.execute(
    """SELECT event_id, event_type, routing_key, payload FROM outbox
       WHERE published_at IS NULL ORDER BY created_at LIMIT 50
       FOR UPDATE SKIP LOCKED"""
).fetchall()
for row in rows:
    event = row["payload"]
    channel.basic_publish(
        exchange=config.EVENTS_EXCHANGE,
        routing_key=row["routing_key"],
        body=json.dumps(event).encode(),
        properties=pika.BasicProperties(
            content_type="application/json",
            # (2) persistent: the message survives a broker restart
            delivery_mode=pika.DeliveryMode.Persistent,
            message_id=str(row["event_id"]),
            # (3) the request's correlation ID travels with the event
            correlation_id=event["correlation_id"],
            type=row["event_type"],
        ),
        # (4) with confirm_delivery() on, this returns only once the broker
        #     has stored the message, and fails if no queue would receive it
        #     (production adds an alternate exchange, so one such event
        #     cannot hold up the others)
        mandatory=True,
    )
    # (5) marked as sent only after the broker's confirmation
    conn.execute("UPDATE outbox SET published_at = now() WHERE event_id = %s",
                 (row["event_id"],))
```

Listing 6: *The idempotent consumer (notification-service/app/consumer.py, annotated excerpt; logging shortened)*

```
def on_message(self, channel, method, properties, body) -> None:
    # (1) log with the correlation ID that came with the event
    token = correlation_id.set(properties.correlation_id)
    try:
        event = json.loads(body)
        notification = build_notification(event)
        # (2) unique index on event_id: save() returns False for a repeat
        if store.save(notification):
            log_event(log, logging.INFO, "notification.sent", ...)
        else:
            log_event(log, logging.INFO, "event.duplicate", ...)
        # (3) acknowledge only after the notification is saved
        channel.basic_ack(method.delivery_tag)
    except (ValueError, MalformedEvent):
        # (4) a message that can never be processed goes to the dead-letter queue
        channel.basic_nack(method.delivery_tag, requeue=False)
    except PyMongoError:
        # (5) the store is down: put the message back and try again later
        time.sleep(2)
        channel.basic_nack(method.delivery_tag, requeue=True)
    finally:
        correlation_id.reset(token)
```

6.2 Dockerfile, Compose file and the path to production

Listing 7: *order-service/Dockerfile (the Notification service's is the same apart from its label)*

```
# syntax=docker/dockerfile:1
# Base image pinned by digest: a moved or re-pushed tag cannot change the build.
ARG PYTHON_IMAGE=python:3.12-
   slim@sha256:a6e34c598f2467ed0e9a8d349809fcd8b5c603269512df273a0bb1784edc11b1

# ---- Stage 1: install the dependencies into a virtual environment ----
FROM ${PYTHON_IMAGE} AS build
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# ---- Stage 2: the runtime image gets only the venv and the code (no pip cache, no build files)
   ----
FROM ${PYTHON_IMAGE}
LABEL org.opencontainers.image.source="https://github.com/imranneta5555/maltashop-microservices" \
      org.opencontainers.image.description="MaltaShop Order service"
ENV PATH="/opt/venv/bin:$PATH" \
    PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1
RUN useradd --uid 10001 --no-create-home --shell /usr/sbin/nologin app
WORKDIR /app
COPY --from=build /opt/venv /opt/venv
COPY app ./app
USER 10001
EXPOSE 8000
HEALTHCHECK --interval=10s --timeout=3s --start-period=15s --retries=3 \
  CMD ["python", "-c", "import urllib.request;
   urllib.request.urlopen('http://localhost:8000/health/ready', timeout=2)"]
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000", "--no-access-log"]
```

The key choices in Listing 7 are:

- **Base image.** `python:3.12-slim` is an official, regularly patched Debian image: small, yet compatible with the ready-built wheels of psycopg and pymongo (Alpine's musl library often forces slow source builds). Both stages pin it to a digest, so a moved tag cannot change the build.
- **Multi-stage build.** The first stage installs the dependencies into a virtual environment; the final image copies only that environment and the application code (Docker, n.d.). No pip cache, build files or tests reach production, so the image is smaller and has less to attack. Dependencies are installed before the code is copied, so a code change reuses the cached layer, and `.dockerignore` keeps tests and `.env` files out.
- **Non-root user.** The process runs as user 10001, so an attacker who escaped the container would not be root on the host; Kubernetes enforces this with `runAsNonRoot` (Listing 3).
- **Health checks.** `HEALTHCHECK` calls `/health/ready`; Compose starts each service only when its database and the broker are healthy (`condition: service_healthy`), and Kubernetes uses `/health/live` and `/health/ready` as separate liveness and readiness probes (Section 4.1).

- **Configuration through environment variables.** Every setting comes from the environment, with development values only in Compose, so no password or environment-specific value is in the image or code, which is what makes "build once, promote everywhere" possible.

Listing 8: *docker-compose.yml (excerpt, passwords shortened; the full file is in Appendix B)*

```
services:
  order-service:
    build: ./order-service
    # CI sets IMAGE_REGISTRY and IMAGE_TAG to run the image it has just built
    image: ${IMAGE_REGISTRY:-local}/maltashop-order-service:${IMAGE_TAG:-dev}
    environment:
      SERVICE_NAME: order-service
      DATABASE_URL: postgresql://order_service:<password>@order-db:5432/orders
      RABBITMQ_URL: amqp://maltashop:<password>@rabbitmq:5672/%2F
    ports: ["8001:8000"]
    depends_on:
      order-db: {condition: service_healthy}
      rabbitmq: {condition: service_healthy}
    networks: [order-net, events-net]   # its own database and the broker only
  order-db:
    image: postgres:16-alpine
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U order_service -d orders"]
    networks: [order-net]
networks:
  order-net:
  notification-net:
  events-net:
```

As Listing 8 shows, separate networks enforce database per service: the Order service cannot even reach the Notification service's database (Figure 6). One command starts all five containers, and Compose waits until each is healthy (Figure 7).

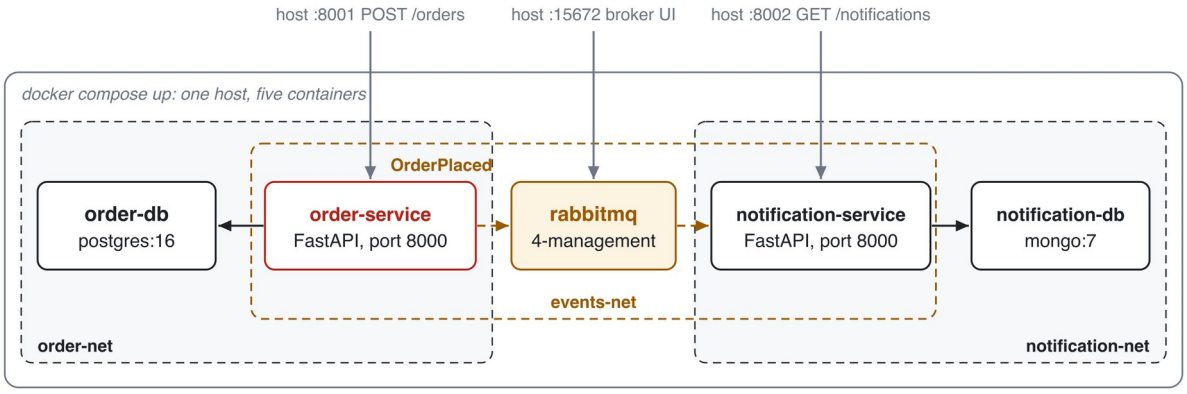


Figure 6: *The prototype under Docker Compose. Each service shares a private network with its own database, and both share an events network with RabbitMQ; only the three ports shown are open to the host.*

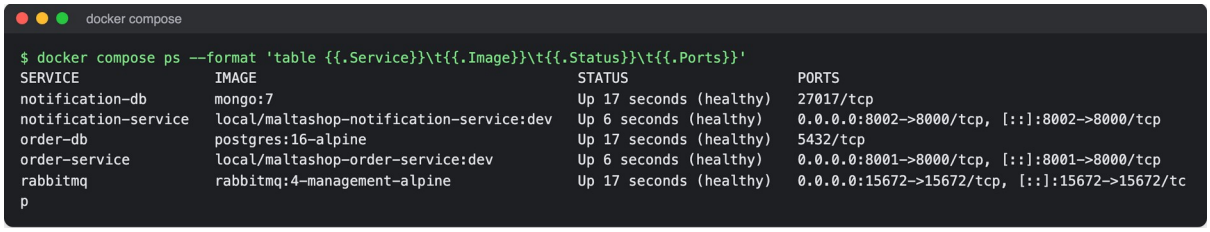


Figure 7: *After* `docker compose up -d --build --wait`*, all five containers are running and healthy.*

Table 15 lists the cloud-native tools needed to take the prototype from a laptop to production, and what each adds.

Table 15: *From laptop to production*

Tool	What it adds
Container registry (Amazon ECR; GHCR in the prototype)	Stores scanned images by commit tag and digest, pulled with the cluster's identity, not a password; signing (cosign) lets the cluster admit only trusted images
Kubernetes (Amazon EKS)	Compose runs on one machine; EKS adds replicas across zones, rolling updates, self-healing and autoscaling (Section 4.1)
Helm charts	One parameterised deployment template per service, replacing Compose
Argo CD (GitOps)	Applies what is in Git to the cluster, with an audit trail and easy rollback
Argo Rollouts	Canary releases with automatic analysis and rollback (Section 3.3)
Managed PostgreSQL, MongoDB and RabbitMQ	Backups, failover and patching without in-house database experts
AWS Secrets Manager with External Secrets	Replaces the development passwords in Compose with managed, rotated secrets
OpenTelemetry Collector and Grafana Cloud	Central logs, metrics, traces and alerts (Section 4.2)
Kong API gateway and Keycloak	Authentication, per-partner scopes and rate limits
Terraform	The cluster, databases and broker defined as reviewable code

6.3 Evidence and reflection

Figures 8 to 13 come from one run of `evidence/capture.py` on a freshly started system.

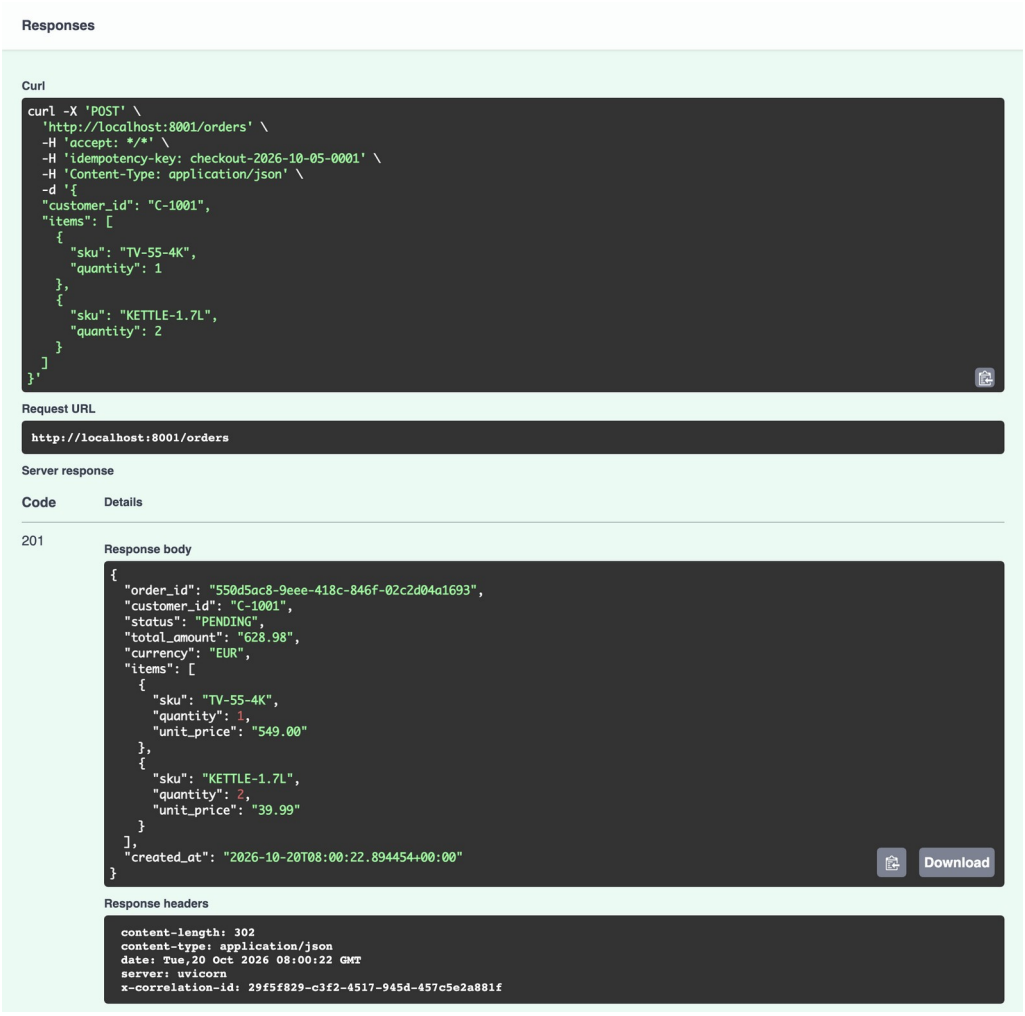


Figure 8: *An order placed through the Order service's API documentation page (Swagger UI): the request, sent with an Idempotency-Key, and the 201 Created response with the order as PENDING, the total priced by the service, and the generated X-Correlation-ID header.*



Figure 9: *The Notification service consumed the OrderPlaced event and stored a notification in its own MongoDB database, carrying the same order ID and correlation ID.*

logs of both services, filtered by one correlation ID

```
$ docker compose logs --no-log-prefix order-service notification-service | grep 29f5f829-c3f2-4517-945d-457c5e
2a881f | jq -r '[.timestamp[11:23], .service, .correlation_id, .event] | @tsv' | sort | column -t -s $'\t'
08:00:22.899  order-service         29f5f829-c3f2-4517-945d-457c5e2a881f  order.placed
08:00:22.900  order-service         29f5f829-c3f2-4517-945d-457c5e2a881f  http.request
08:00:23.805  order-service         29f5f829-c3f2-4517-945d-457c5e2a881f  event.published
08:00:23.809  notification-service  29f5f829-c3f2-4517-945d-457c5e2a881f  event.received
08:00:23.843  notification-service  29f5f829-c3f2-4517-945d-457c5e2a881f  notification.sent
```

Figure 10: *Log output of both services filtered by one correlation ID: the request, the outbox publish, and the event received and processed by the Notification service.*

retrying the same request

```
$ curl -s -i -X POST http://localhost:8001/orders -H 'Content-Type: application/json' -H 'Idempotency-Key: che
ckout-2026-10-05-0001' -d '{"customer_id": "C-1001", "items": [{"sku": "TV-55-4K", "quantity": 1}, {"sku": "KE
TTLE-1.7L", "quantity": 2}]}' | grep -E -i '^HTTP|^idempotent|order_id' | cut -c1-100
HTTP/1.1 200 OK
idempotent-replayed: true
{"order_id":"550d5ac8-9eee-418c-846f-02c2d04a1693","customer_id":"C-1001","status":"PENDING","total_

$ curl -s 'http://localhost:8002/notifications?order_id=550d5ac8-9eee-418c-846f-02c2d04a1693' | jq length
1
```

Figure 11: *Retrying the same request with the same Idempotency-Key returns the original order (200 OK, Idempotent-Replayed), and there is still exactly one notification.*

Notification service stopped: checkout still works

```
$ docker compose stop notification-service
 Container maltashop-notification-service-1 Stopping
 Container maltashop-notification-service-1 Stopped

$ curl -s -o /dev/null -w 'HTTP %{http_code}\n' -X POST http://localhost:8001/orders -H 'Content-Type: applica
tion/json' -H 'Idempotency-Key: outage-test-0001' -d '{"customer_id": "C-1001", "items": [{"sku": "TV-55-4K",
"quantity": 1}, {"sku": "KETTLE-1.7L", "quantity": 2}]}'
HTTP 201

$ sleep 3; docker compose exec -T rabbitmq rabbitmqctl list_queues --quiet name messages_ready | column -t
name                                  messages_ready
notification-service.order-placed.dlq  0
notification-service.order-placed      1
```

Figure 12: *Resilience test: with the Notification service stopped, an order is still accepted (HTTP 201) and its event waits in the Notification service's queue.*

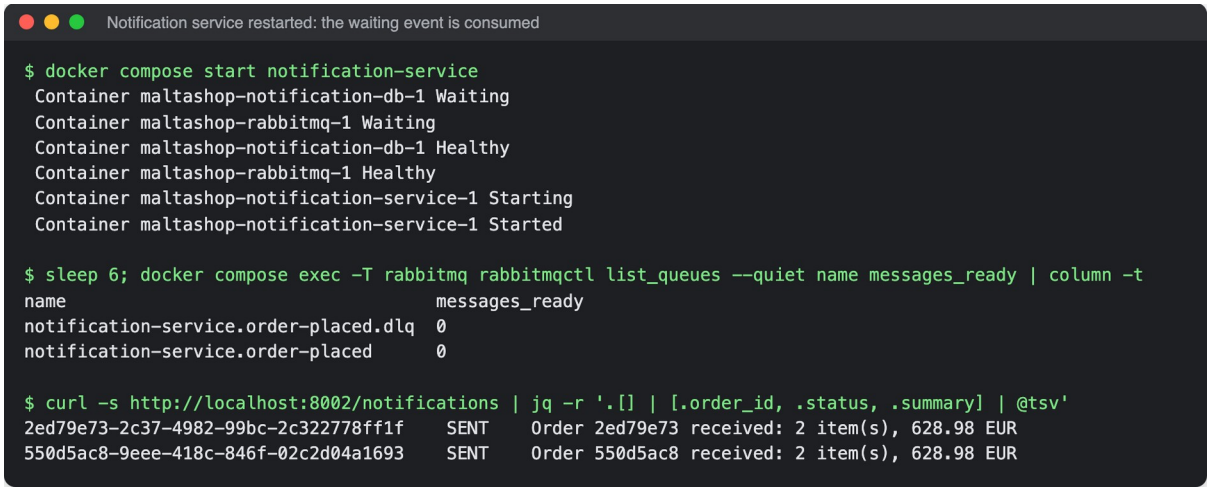


Figure 13: *After the Notification service restarts, the waiting event is consumed: the queue is empty and both orders have a notification.*

What the prototype proves. Two services with separate databases and networks cooperate only through a REST call and an event, which is the core of the design. Figures 12 and 13 show that the March incident cannot recur in the same form: with Notification stopped, orders were still accepted, the event waited in RabbitMQ and was processed when Notification returned. Figure 11 shows that a retried checkout creates no duplicate, and Figure 10 shows one correlation ID following an order across both services, as Section 4.2 requires. The CI pipeline tests the exact images it built.

Where it falls short. It has two services, not eight: without Inventory and Payment, the saga's compensations are designed (Figure 5) but never executed. There is no gateway, authentication or tracing, only correlation IDs. The outbox is polled every second, so an event can wait up to a second before it is published (Figure 10). Every container is a single instance, and the passwords are development defaults.

What I would change before production. Add Inventory and Payment with their compensations, and test the failure paths automatically; replace polling with PostgreSQL LISTEN/NOTIFY or change-data capture; run schema changes as versioned migrations; add OpenTelemetry tracing, the gateway and Keycloak; move secrets into a secrets manager; and run the load test at ten times normal traffic on Kubernetes before the first sale.

References

Beyer, B., Murphy, N.R., Rensin, D.K., Kawahara, K. and Thorne, S. (eds) (2018) *The Site Reliability Workbook: Practical Ways to Implement SRE*. Sebastopol, CA: O'Reilly Media.

Burns, B., Beda, J., Hightower, K. and Evenson, L. (2022) *Kubernetes: Up and Running*. 3rd edn. Sebastopol, CA: O'Reilly Media.

Cohn, M. (2009) *Succeeding with Agile: Software Development Using Scrum*. Upper Saddle River, NJ: Addison-Wesley.

Conway, M.E. (1968) 'How do committees invent?', *Datamation*, 14(4), pp. 28–31.

Docker (n.d.) *Multi-stage builds*. Available at: https://docs.docker.com/build/building/multi-stage/ (Accessed: 5 October 2026).

Erl, T. (2005) *Service-Oriented Architecture: Concepts, Technology, and Design*. Upper Saddle River, NJ: Prentice Hall.

European Parliament and Council of the European Union (2016) 'Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation)', *Official Journal of the European Union*, L 119, 4 May, pp. 1–88.

Forsgren, N., Humble, J. and Kim, G. (2018) *Accelerate: The Science of Lean Software and DevOps*. Portland, OR: IT Revolution Press.

Fowler, M. (2004) *StranglerFigApplication*. Available at: https://martinfowler.com/bliki/StranglerFigApplication.html (Accessed: 5 October 2026).

Fowler, M. (2010) *BlueGreenDeployment*. Available at: https://martinfowler.com/bliki/BlueGreenDeployment.html (Accessed: 5 October 2026).

Fowler, M. (2015) *MonolithFirst*. Available at: https://martinfowler.com/bliki/MonolithFirst.html (Accessed: 5 October 2026).

Garcia-Molina, H. and Salem, K. (1987) 'Sagas', *ACM SIGMOD Record*, 16(3), pp. 249–259.

Hammant, P. (2017) *Trunk Based Development*. Available at: https://trunkbaseddevelopment.com (Accessed: 5 October 2026).

Humble, J. and Farley, D. (2010) *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Upper Saddle River, NJ: Addison-Wesley.

Istio (n.d.) *The Istio service mesh*. Available at: https://istio.io/latest/about/service-mesh/ (Accessed: 5 October 2026).

Kubernetes (n.d.a) *Configure liveness, readiness and startup probes*. Available at: https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/ (Accessed: 5 October 2026).

Kubernetes (n.d.b) *Horizontal Pod Autoscaling*. Available at: https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/ (Accessed: 5 October 2026).

Lewis, J. and Fowler, M. (2014) *Microservices: a definition of this new architectural term*. Available at: https://martinfowler.com/articles/microservices.html (Accessed: 5 October 2026).

Newman, S. (2021) *Building Microservices: Designing Fine-Grained Systems*. 2nd edn. Sebastopol, CA: O'Reilly Media.

Nygard, M. (2011) *Documenting architecture decisions*. Available at: https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions (Accessed: 5 October 2026).

Nygard, M.T. (2018) *Release It! Design and Deploy Production-Ready Software*. 2nd edn. Raleigh, NC: Pragmatic Bookshelf.

OpenTelemetry (n.d.) *OpenTelemetry documentation*. Available at: https://opentelemetry.io/docs/ (Accessed: 5 October 2026).

Pact Foundation (n.d.) *Introduction to Pact*. Available at: https://docs.pact.io/ (Accessed: 5 October 2026).

PCI Security Standards Council (2022) *Payment Card Industry Data Security Standard: Requirements and Testing Procedures, v4.0*. Wakefield, MA: PCI Security Standards Council.

RabbitMQ (n.d.) *Consumer acknowledgements and publisher confirms*. Available at: https://www.rabbitmq.com/docs/confirms (Accessed: 5 October 2026).

Richardson, C. (2018) *Microservices Patterns: With Examples in Java*. Shelter Island, NY: Manning.

Richardson, C. (n.d.) *Pattern: Transactional outbox*. Available at: https://microservices.io/patterns/data/transactional-outbox.html (Accessed: 5 October 2026).

Rotem-Gal-Oz, A. (2006) *Fallacies of distributed computing explained*. Available at: https://www.rgoarchitects.com/Files/fallacies.pdf (Accessed: 5 October 2026).

Sato, D. (2014) *CanaryRelease*. Available at: https://martinfowler.com/bliki/CanaryRelease.html (Accessed: 5 October 2026).

Skelton, M. and Pais, M. (2019) *Team Topologies: Organizing Business and Technology Teams for Fast Flow*. Portland, OR: IT Revolution Press.

Vocke, H. (2018) *The Practical Test Pyramid*. Available at: https://martinfowler.com/articles/practical-test-pyramid.html (Accessed: 5 October 2026).

Vogels, W. (2009) 'Eventually consistent', *Communications of the ACM*, 52(1), pp. 40–44.

Wiggins, A. (2017) *The Twelve-Factor App*. Available at: https://12factor.net/ (Accessed: 5 October 2026).

Appendix A: Git Repository

The prototype, its tests, the CI workflow, the diagrams and the scripts that build this report are in a public GitHub repository, shared with the lecturer:
https://github.com/imranneta5555/maltashop-microservices

To run the prototype, clone the repository and start it with Docker Compose. The README explains the endpoints and every environment variable.

```
git clone https://github.com/imranneta5555/maltashop-microservices.git
cd maltashop-microservices
docker compose up --build -d --wait
python3 scripts/smoke_test.py
```

The commit history below, oldest first, shows the prototype being built step by step (it was taken when this report was built).

```
f47524f  2026-10-05 10:02  project skeleton and readme outline
56b5272  2026-10-05 10:04  order service: order rules, pricing and OrderPlaced event with unit
  tests
bcc2e31  2026-10-05 10:06  order service: POST /orders with PostgreSQL, idempotency key and outbox
  table
1b55810  2026-10-05 10:10  order service: outbox relay publishes OrderPlaced to RabbitMQ with
  publisher confirms
9ae0ca9  2026-10-05 10:11  notification service: idempotent OrderPlaced consumer with MongoDB store
  and dead-letter queue
b066a7e  2026-10-05 10:14  dockerfiles for both services and docker compose for the whole system
deaa8e7  2026-10-05 10:15  OrderPlaced contract with provider and consumer contract tests, end-to-
  end smoke test
30dacf7  2026-10-05 10:16  ci: test, build-once image push to GHCR, and integration stage on the
  built images
c727c9d  2026-10-05 10:16  readme: what the system is, how to run it, endpoints and environment
  variables
55423a0  2026-10-05 10:22  ci: pin actions to commit SHAs, move to node24 releases, pin runner to
  ubuntu-24.04
ae9e7fa  2026-10-05 10:25  diagrams: architecture, ci/cd pipeline, observability, saga and
  prototype topology
df25e1b  2026-10-05 10:29  evidence: demo capture script, transcripts and screenshots of the
  running prototype
869c520  2026-10-05 10:54  order service: wrap long lines that the report quotes
cb1f0f6  2026-10-05 10:54  evidence: larger terminal text, tighter crops and the ci run screenshot
11193e5  2026-10-05 10:54  docs: architecture decision records; readme covers docs, evidence and
  report
343bb64  2026-10-05 10:54  report: sections 1 to 6, references and the build scripts
bd1815b  2026-10-05 10:54  report built (pdf and word)
343b191  2026-10-05 10:57  report rebuilt with the latest passing ci run
e6e26d6  2026-10-05 11:36  report rebuilt after the history rewrite
971ec6a  2026-10-05 11:48  report: web addresses are clickable links
c932b8e  2026-10-05 11:48  report rebuilt with clickable links
cff5406  2026-10-20 14:00  config: defaults hold no passwords (12-factor III)
3a50180  2026-10-20 14:00  compose: named volume for RabbitMQ so queued events survive a re-created
  container
56203cd  2026-10-20 14:00  docker: base image pinned by digest; .dockerignore keeps .env files and
  caches out
4df861a  2026-10-20 14:00  ci: trivy scan gate before push; images built only for main
5a80ca5  2026-10-20 14:00  diagrams: pipeline labels use the lecture's test levels and the gitops
  step
6125c84  2026-10-20 14:00  evidence: re-captured after the prototype changes
021220b  2026-10-20 14:00  report: aligned with the lecture slides; shorter appendix and no forced
  section breaks
```

Appendix B: Docker Compose File

The full `docker-compose.yml` quoted in Listing 8. The Dockerfile and the CI workflow are in Listings 7 and 2; all other source code is in the repository.

```
# MaltaShop prototype: start everything with `docker compose up --build`.
# Every value can be overridden from a .env file (see .env.example); the defaults are for a laptop
   only.
name: maltashop

services:
  # ---------- Order service and the database only it can reach ----------
  order-service:
    build: ./order-service
    image: ${IMAGE_REGISTRY:-local}/maltashop-order-service:${IMAGE_TAG:-dev}
    environment:
      SERVICE_NAME: order-service
      DATABASE_URL: postgresql://order_service:${ORDER_DB_PASSWORD:-order_dev_password}@order-
   db:5432/orders
      RABBITMQ_URL: amqp://${RABBITMQ_USER:-maltashop}:${RABBITMQ_PASSWORD:-maltashop_dev_password}
   @rabbitmq:5672/%2F
      EVENTS_EXCHANGE: maltashop.events
      OUTBOX_POLL_SECONDS: "1"
      LOG_LEVEL: ${LOG_LEVEL:-INFO}
    ports:
      - "8001:8000"
    depends_on:
      order-db:
        condition: service_healthy
      rabbitmq:
        condition: service_healthy
    networks: [order-net, events-net]
    restart: unless-stopped

  order-db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: orders
      POSTGRES_USER: order_service
      POSTGRES_PASSWORD: ${ORDER_DB_PASSWORD:-order_dev_password}
    volumes:
      - order-db-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U order_service -d orders"]
      interval: 5s
      timeout: 3s
      retries: 12
    networks: [order-net]

  # ---------- Notification service and the database only it can reach ----------
  notification-service:
    build: ./notification-service
    image: ${IMAGE_REGISTRY:-local}/maltashop-notification-service:${IMAGE_TAG:-dev}
    environment:
      SERVICE_NAME: notification-service
      MONGO_URL: mongodb://notification_service:${NOTIFICATION_DB_PASSWORD:-
   notification_dev_password}@notification-db:27017/?authSource=admin
      MONGO_DB: notifications
      RABBITMQ_URL: amqp://${RABBITMQ_USER:-maltashop}:${RABBITMQ_PASSWORD:-maltashop_dev_password}
   @rabbitmq:5672/%2F
      EVENTS_EXCHANGE: maltashop.events
      QUEUE_NAME: notification-service.order-placed
      LOG_LEVEL: ${LOG_LEVEL:-INFO}
    ports:
      - "8002:8000"
    depends_on:
      notification-db:
        condition: service_healthy
      rabbitmq:
        condition: service_healthy
    networks: [notification-net, events-net]
    restart: unless-stopped

  notification-db:
```

```
    image: mongo:7
    environment:
      MONGO_INITDB_ROOT_USERNAME: notification_service
      MONGO_INITDB_ROOT_PASSWORD: ${NOTIFICATION_DB_PASSWORD:-notification_dev_password}
    volumes:
      - notification-db-data:/data/db
    healthcheck:
      test: ["CMD", "mongosh", "--quiet", "--eval", "db.adminCommand('ping').ok"]
      interval: 5s
      timeout: 5s
      retries: 12
    networks: [notification-net]

  # ---------- Message broker shared by both services ----------
  rabbitmq:
    image: rabbitmq:4-management-alpine
    hostname: rabbitmq   # a fixed node name, so the volume below is reused after re-creation
    environment:
      RABBITMQ_DEFAULT_USER: ${RABBITMQ_USER:-maltashop}
      RABBITMQ_DEFAULT_PASS: ${RABBITMQ_PASSWORD:-maltashop_dev_password}
    volumes:
      - rabbitmq-data:/var/lib/rabbitmq   # queued events survive a re-created container
    ports:
      - "15672:15672"   # management UI: http://localhost:15672
    healthcheck:
      test: ["CMD", "rabbitmq-diagnostics", "-q", "ping"]
      interval: 10s
      timeout: 5s
      retries: 12
    networks: [events-net]

# Separate networks: each service can reach its own database and the broker, but not the
# other service's database. "Database per service" is enforced, not just agreed.
networks:
  order-net:
  notification-net:
  events-net:

volumes:
  order-db-data:
  notification-db-data:
  rabbitmq-data:
```